THE UNIVERSITY OF
Azad Jammu and Kashmir
Department of Software Engineering · Muzaffarabad

LAB TASK

02

DIGITAL IMAGE PROCESSING LAB

Digital Images, Pixels, Color, and Basic Transformations

SUBMITTED BY Syeda Khadija Hassan	ROLL NO 2024-SE-09
LAB INSTRUCTOR Engr. Fahad Nisar	DEPARTMENT Software Engineering
SESSION 2024-28 (SE)	SEMESTER Fifth
ASSIGNED ON October 22, 2026	DUE DATE October 29, 2026, 11:59 PM

LAB REPORT | FIFTH SEMESTER | SESSION 2024-28

1. Introduction

A digital image is a grid of numbers. Each number (a pixel) records how bright one tiny point is, and a color image keeps three such numbers per point, one for each color channel. In this lab I used Python with NumPy, OpenCV and Matplotlib to load a personally selected color image, look at its raw pixel values, split it into channels, crop and resize part of it, study its brightness histogram, and finally demonstrate what happens when the number of intensity levels (the bit depth) is reduced.

2. Objectives

- Set up an isolated Python virtual environment and install numpy, opencv-python and matplotlib.
- Load an image with cv.imread() and inspect its shape, data type and pixel values using [row, column] indexing.
- Convert between BGR, RGB and grayscale, separate the color channels and display them correctly.
- Crop an exact 100 × 100 region, resize it to 200 × 200 and save it with cv.imwrite().
- Plot and interpret a 256-bin grayscale histogram.
- Demonstrate 1-, 2-, 4- and 8-bit intensity quantization and explain resolution versus bit depth.

3. Requirements

- **Software:** Python 3, Visual Studio Code, and the packages numpy, opencv-python and matplotlib.
- **Input image:** one personally selected color JPG/PNG image of at least 300 × 300 pixels (the instructor's demonstration image was not used).
- **Project folder:** dip-lab-02, containing lab02.py, the input image, resized_crop.png, grayscale_histogram.png and .gitignore.

4. Steps

Step 1: Create the Project and Virtual Environment

I created the folder dip-lab-02, opened it in VS Code and created a virtual environment inside it, so that the packages for this lab stay separate from the rest of my system. After activating it I installed the three libraries, checked that they import correctly, and added .venv/ to .gitignore so the environment is never submitted.

COMMANDS USED (COMMAND PROMPT):

```
mkdir dip-lab-02
cd dip-lab-02
python -m venv .venv
.venv\Scripts\activate.bat
pip install numpy opencv-python matplotlib
python -c "import sys, numpy, cv2, matplotlib; print(sys.executable); print(numpy.__version__,
cv2.__version__, matplotlib.__version__)"
echo .venv/ > .gitignore
```

VERIFICATION CODE (FIRST LINES OF THE OUTPUT OF LAB02.PY):

```
banner("STEP 1: Environment check")
print("Python executable :", sys.executable)
print("NumPy version     :", np.__version__)
print("OpenCV version    :", cv.__version__)
print("Matplotlib version:", plt.matplotlib.__version__)
```

EVIDENCE 1.1: VIRTUAL ENVIRONMENT CREATED AND ACTIVATED, PACKAGES INSTALLED

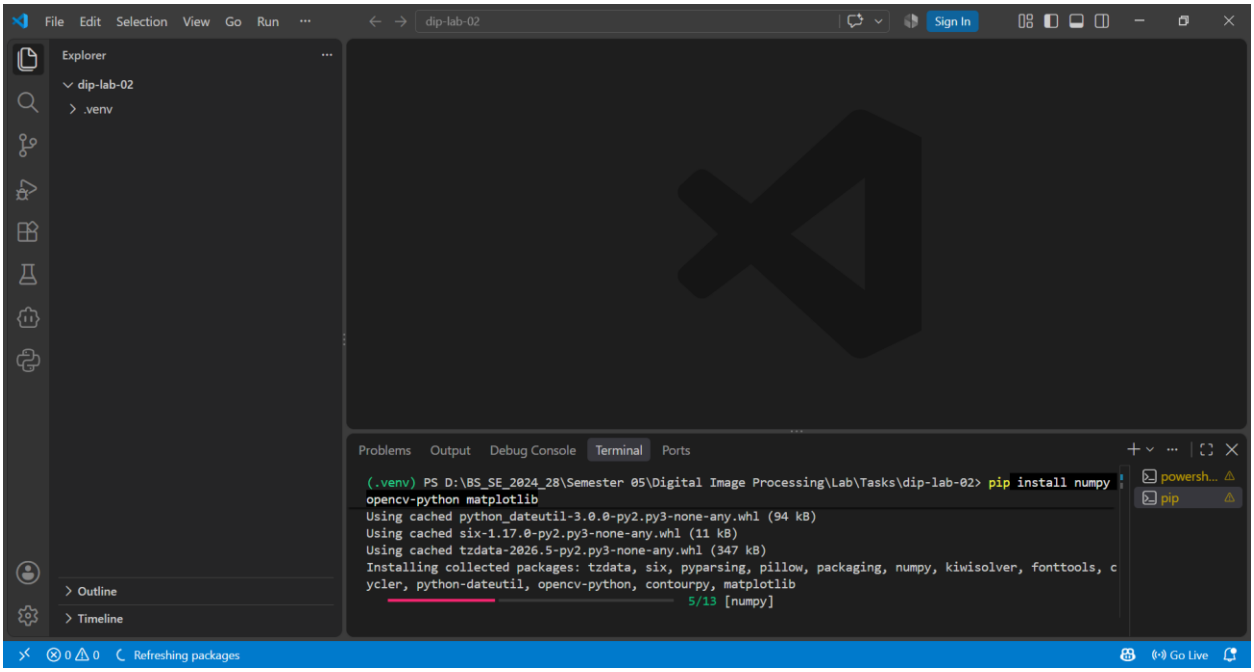


Figure 1.1: Virtual environment and package installation

EVIDENCE 1.2: IMPORTS AND PYTHON EXECUTABLE VERIFIED

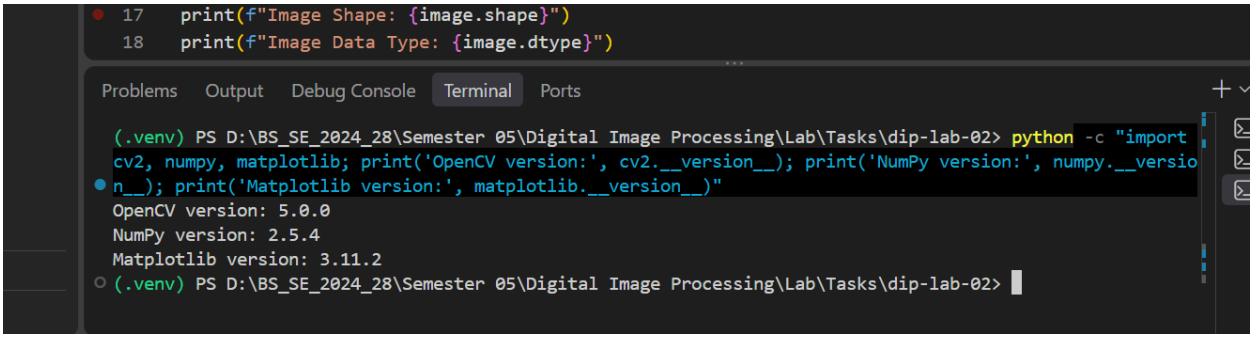


Figure 1.2: Import and Python executable verification

Step 2: Load the Image

I placed my chosen image in dip-lab-02 and imported OpenCV as cv, NumPy as np and Matplotlib's pyplot as plt. cv.imread() returns None instead of raising an error when a file is missing, so I check for None and raise FileNotFoundError myself. Then I print the shape and data type.

CODE:

```
import os
import sys


import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

```
img = cv.imread(IMAGE_PATH)          # OpenCV loads color images as BGR
if img is None:
    raise FileNotFoundError(f"Image not found or unreadable: {IMAGE_PATH}")


print("Image shape (H, W, C):", img.shape)
print("Image data type      :", img.dtype)
```

OUTPUT:

```
--- Step 2: Image Loading ---
Image Shape: (920, 736, 3)
Image Data Type: uint8
```

Figure 2.1: Image shape and data type

The shape is reported as (height, width, channels). The data type uint8 means every value is an 8-bit integer from 0 to 255.

Step 3: Inspect the Image

Here I read the height, width and channel count from img.shape. NumPy indexes pixels as [row, column], which means [y, x] (the row comes first). The top-left pixel is img[0, 0] and the center pixel is img[height // 2, width // 2]. Because OpenCV stores color in BGR order, each printed pixel is shown as (Blue, Green, Red). The global and per-channel minimum and maximum values are computed with .min() and .max().

CODE:

```
height, width, channels = img.shape
print("Height  :", height, "pixels")
print("Width   :", width, "pixels")
print("Channels:", channels)


center_row, center_col = height // 2, width // 2
print("Top-left pixel [0, 0] (B, G, R)              :", img[0, 0])
print(f"Center pixel [{center_row}, {center_col}] (B, G, R):", img[center_row, center_col])


print("Global minimum value:", img.min())
```

```
print("Global maximum value:", img.max())
for index, name in enumerate(("Blue", "Green", "Red")):   # BGR order
    print(f"{name:>5} channel -> min: {img[:, :, index].min()}, max: {img[:, :, index].max()}")
```

OUTPUT:

```
--- Step 3: Image Inspection ---
Height: 920 pixels
Width: 736 pixels
Number of Channels: 3
Top-Left Pixel (BGR) at [0, 0]: [196 156 121]
Center Pixel (BGR) at [460, 368]: [83 77 72]

Global Minimum Pixel Value: 0
Global Maximum Pixel Value: 255
Blue Channel Min/Max: 0 / 255
Green Channel Min/Max: 0 / 255
Red Channel Min/Max: 0 / 255
```

Figure 3.1: Image dimensions, pixel values and value ranges

Step 4: Convert Colors and Separate Channels

OpenCV loads color as BGR, but Matplotlib expects RGB. I therefore converted the image with cv.cvtColor(img, cv.COLOR_BGR2RGB), made a grayscale copy with cv.COLOR_BGR2GRAY, and split the RGB image into its red, green and blue channels. The 2 × 3 figure shows the RGB image, the three channels (gray colormap), the grayscale image and, for comparison, the original BGR array shown without conversion. Finally the center pixel's RGB value is converted to a HEX color code.

CODE:

```
rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
red, green, blue = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]


fig, axes = plt.subplots(2, 3, figsize=(12, 8))
panels = [
    (rgb, "RGB Image", None),
    (red, "Red Channel", "gray"),
    (green, "Green Channel", "gray"),
    (blue, "Blue Channel", "gray"),
    (gray, "Grayscale Image", "gray"),
    (img, "Unconverted BGR Display", None),   # wrong colors on purpose
]
for ax, (data, title, cmap) in zip(axes.flat, panels):
    ax.imshow(data, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()


r, g, b = (int(v) for v in rgb[center_row, center_col])
```

```
hex_value = f"#{r:02X}{g:02X}{b:02X}"
print(f"Center pixel RGB value: ({r}, {g}, {b})")
print("Center pixel HEX value:", hex_value)
```

OUTPUT FIGURE:

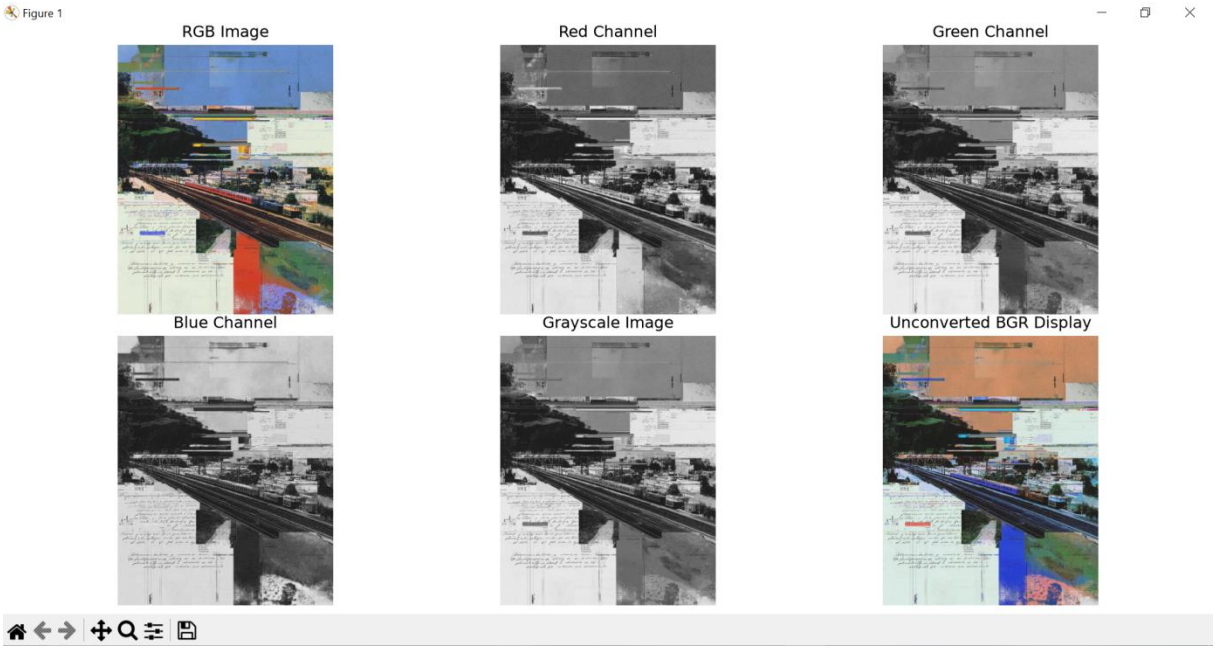


Figure 4.1: RGB, red, green, blue, grayscale and unconverted BGR display

OUTPUT (HEX VALUE):

```
--- Step 4: Color Analysis ---
Center Pixel (RGB): [72 77 83]
Center Pixel HEX Code: #484d53
```

Figure 4.2: Center pixel RGB and HEX value

OBSERVATIONS:

- In the unconverted BGR panel the colors are swapped: the blue sky looks orange and the red block at the bottom looks blue. Red and blue have changed places because Matplotlib reads the first channel as red.
- The red block at the bottom is bright in the Red channel and dark in the Blue channel, while the blue sky is brighter in the Blue channel than in the Red channel. A bright area in a channel means that color contributes strongly there.
- The grayscale image is a single-channel brightness image. It keeps the structure of the picture (train, tracks, paper) but all color information is gone.
- The HEX value is the same RGB triple written in base 16, two digits each for red, green and blue (#RRGGBB).

Step 5: Crop, Resize, and Save

I cut an exact 100 × 100 block out of the RGB image with NumPy slicing, image[y_start:y_end, x_start:x_end] (rows first, then columns), and checked that its shape is (100, 100, 3). cv.resize() then enlarged it to 200 × 200 (note that cv.resize() takes the size as (width, height)). The original, crop and resized crop are shown side by side. Since cv.imwrite() expects BGR, I converted the resized RGB crop back to BGR before saving it as resized_crop.png, and printed whether the save succeeded.

CODE:

```
y_start, x_start = CROP_Y_START, CROP_X_START
y_end, x_end = y_start + CROP_SIZE, x_start + CROP_SIZE

crop = rgb[y_start:y_end, x_start:x_end]
print("Crop shape:", crop.shape)
if crop.shape[:2] != (CROP_SIZE, CROP_SIZE):
    raise ValueError("Crop region falls outside the image; adjust the start coordinates.")

resized = cv.resize(crop, RESIZE_TO)
print("Resized shape:", resized.shape)

fig, axes = plt.subplots(1, 3, figsize=(12, 5))
for ax, (data, title) in zip(axes, [(rgb, "Original Image"),
                                    (crop, "100x100 Crop"),
                                    (resized, "200x200 Resized")]):
    ax.imshow(data)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()

resized_bgr = cv.cvtColor(resized, cv.COLOR_RGB2BGR)
saved = cv.imwrite("resized_crop.png", resized_bgr)
print("cv.imwrite() saved resized_crop.png successfully:", saved)
```

OUTPUT FIGURE:

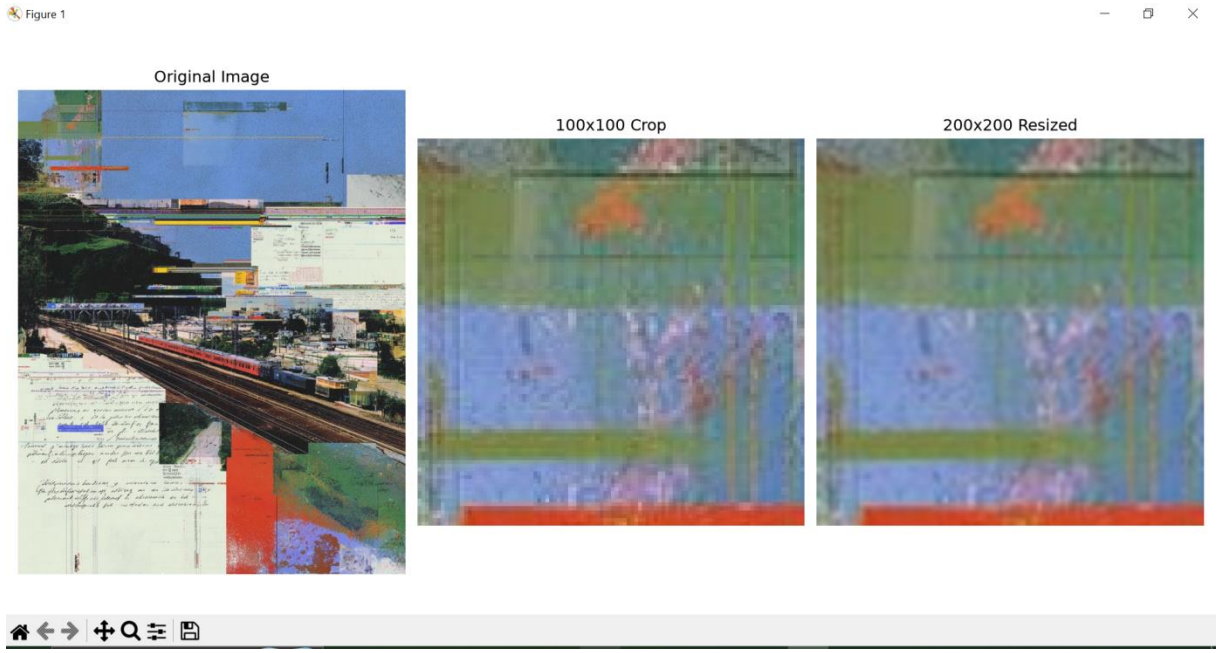


Figure 5.1: Original image, 100 × 100 crop and 200 × 200 resized crop

OUTPUT (SHAPES AND SAVE STATUS):

```
--- Step 5: Crop and Resize ---
Cropped Image Shape: (100, 100, 3)
Did cv.imwrite save the file successfully? True
```

Figure 5.2: Crop shape, resized shape and save confirmation

The resized crop is larger but not sharper. Resizing only creates new pixels by interpolating between existing neighbors, so it makes the picture look smoother and blurrier, but no new detail appears.

Step 6: Create a Histogram

The grayscale image was flattened into a one-dimensional array of pixel values, and plt.hist() was used with 256 bins over the range 0 to 256, so that every bin matches exactly one intensity level (0 to 255). The plot has a title and axis labels and was saved as grayscale_histogram.png before being displayed. I also print the mean intensity and the percentage of dark, mid and bright pixels as numerical support.

CODE:

```
flat = gray.flatten()
print("Mean grayscale intensity:", round(float(flat.mean()), 2))
print("Dark pixels   (0-84)   :", f"{np.mean(flat < 85) * 100:.1f}%")
print("Mid pixels    (85-169) :", f"{np.mean((flat >= 85) & (flat < 170)) * 100:.1f}%")
print("Bright pixels (170-255):", f"{np.mean(flat >= 170) * 100:.1f}%")


plt.figure(figsize=(12, 6))
```

```
plt.hist(flat, bins=256, range=(0, 256), color="#333333")
plt.title("Grayscale Image Histogram")
plt.xlabel("Pixel Intensity (0-255)")
plt.ylabel("Frequency (Number of Pixels)")
plt.savefig("grayscale_histogram.png", dpi=150, bbox_inches="tight")
print("Histogram saved as grayscale_histogram.png:", os.path.exists("grayscale_histogram.png"))
plt.show()
```

OUTPUT FIGURE:

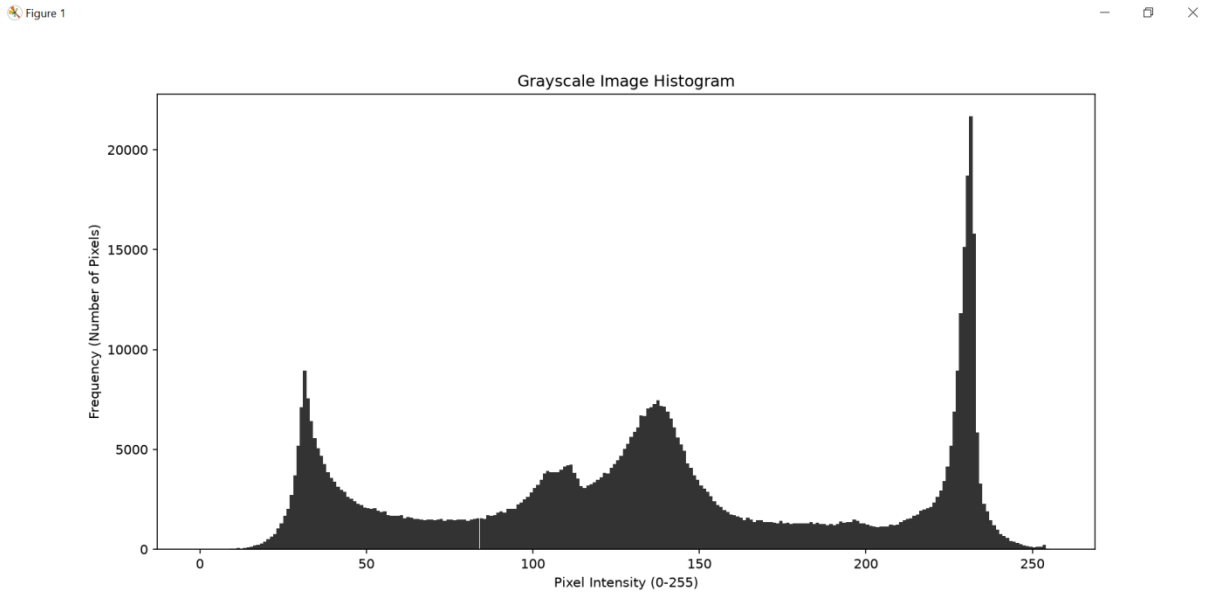


Figure 6.1: Grayscale image histogram (256 bins)

OUTPUT (NUMERICAL SUMMARY):

```
--- Step 6: Histogram ---
Histogram saved as 'grayscale_histogram.png'.
Explanation: The histogram indicates that the image is predominantly bright, which is evidenced by the ma
ssive spike in pixel frequency on the far right side (around intensity levels 220-230). There is also a n
oticeable presence of darker pixels (a smaller peak around level 30-40), but the mid-tones are relatively
 sparse. Overall, the image exhibits high contrast with a strong leaning towards bright highlights rather
 than being mainly dark or perfectly balanced.
```

Figure 6.2: Histogram statistics and save confirmation

INTERPRETATION:

The image is neither mainly dark nor mainly bright. The histogram has three groups: a dark peak near intensity 30–35, a broad mid-tone hump around 135, and a very tall, narrow peak near 230 (over 20,000 pixels). The large bright peak, probably the pale paper areas of the picture, pulls the image slightly toward the bright side, but since the dark and mid-tone groups are also strong and there are almost no pixels at exactly 0 or 255, I would call the image balanced and high in contrast, with a mild bright bias.

Step 7: Demonstrate Bit Depth

I built an 8-bit horizontal gradient with the values 0 to 255 from left to right. A bit depth of n bits allows 2^n intensity levels, so the gradient was quantized to 1 bit (2 levels), 2 bits (4 levels) and 4 bits (16 levels). The quantize() function groups the 256 values into 2^n equal bins and then spreads the results back over 0–255, so that the darkest level is 0 and the brightest is 255. The original and the three quantized gradients are displayed in one figure and the number of unique values in each is printed.

CODE:

```
gradient = np.tile(np.arange(256, dtype=np.uint8), (100, 1))   # 0..255 left to right



def quantize(image, bits):
    levels = 2 ** bits
    step = 256 // levels                        # width of each input bin
    return ((image // step) * (255 // (levels - 1))).astype(np.uint8)



versions = [
    (gradient, "Original 8-bit (256 levels)"),
    (quantize(gradient, 1), "1-bit (2 levels)"),
    (quantize(gradient, 2), "2-bit (4 levels)"),
    (quantize(gradient, 4), "4-bit (16 levels)"),
]
fig, axes = plt.subplots(4, 1, figsize=(12, 8))
for ax, (data, title) in zip(axes, versions):
    ax.imshow(data, cmap="gray", vmin=0, vmax=255, aspect="auto")
    ax.set_title(title)
    ax.axis("off")
    print(f"{title:<30} unique values: {len(np.unique(data))}")
plt.tight_layout()
plt.show()
```

OUTPUT FIGURE:

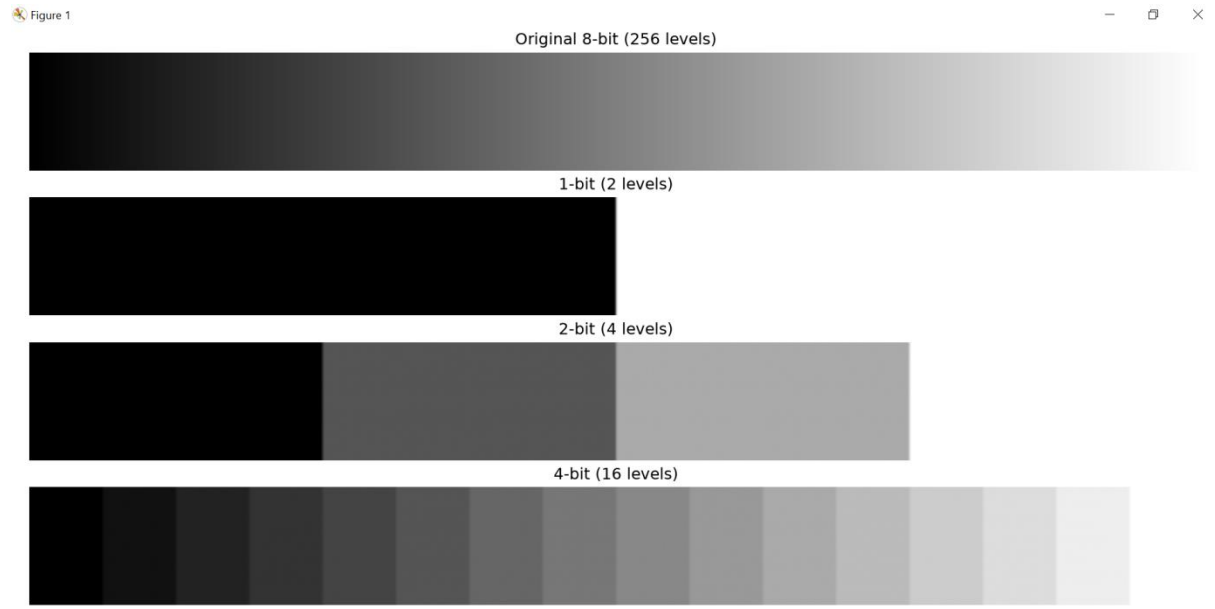


Figure 7.1: Original 8-bit gradient and 1-, 2- and 4-bit quantized versions

OUTPUT (UNIQUE VALUES):

```
--- Step 7: Bit Depth ---
Unique values in 1-bit: 2
Unique values in 2-bit: 4
Unique values in 4-bit: 16
```

Figure 7.2: Unique intensity values in each gradient

The smooth 8-bit gradient turns into visible bands as the bit depth falls: 2 bands at 1 bit, 4 at 2 bits and 16 at 4 bits. All four gradients still have the same width in pixels, so only the number of available brightness levels has changed. The brightest level is pure white (255), which blends into the white background of the figure, so the last band appears to be missing at the right end of each bar.

Step 8: Complete the Project

I ran lab02.py from the activated environment, fixed the errors that appeared, and confirmed that the script runs from start to finish. The project folder holds all required files, and the virtual environment was deactivated afterwards with the deactivate command. The .venv folder is not part of the submission.

COMMANDS USED:

```
python lab02.py
deactivate
```

.GITIGNORE CONTENTS:

```
.venv/
```

EVIDENCE 8.1: PROJECT FOLDER CONTENTS AND DEACTIVATE

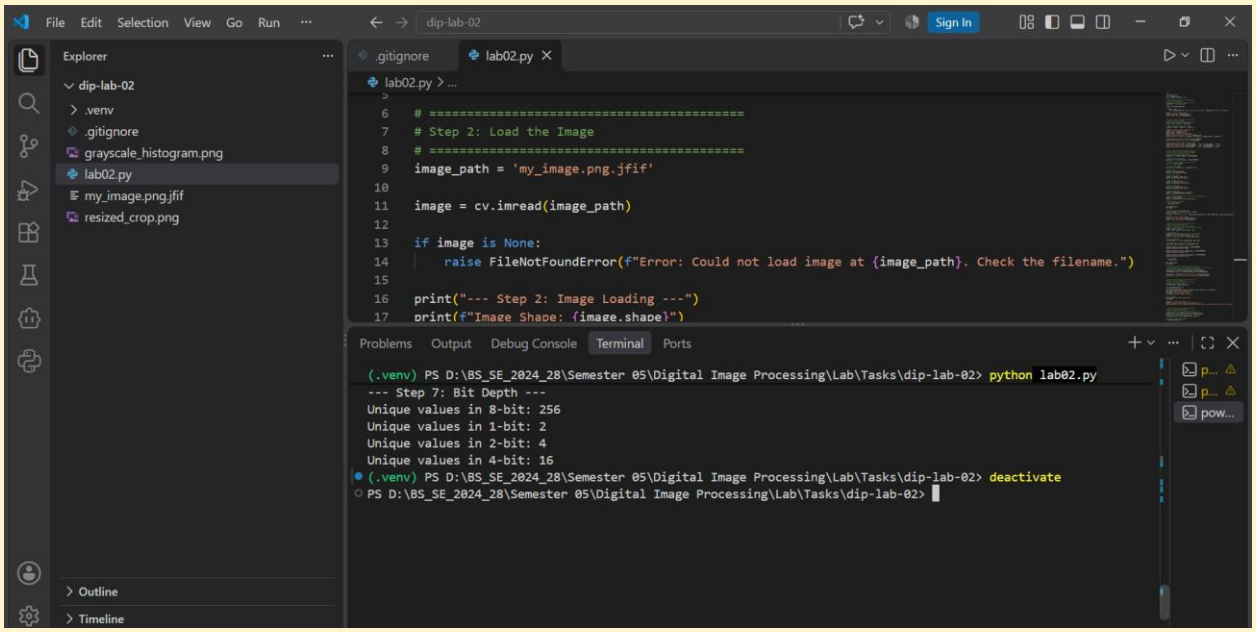


Figure 8.1: Project files and deactivated environment

5. Concepts in My Own Words

5.1 BGR versus RGB

Both describe a pixel with three numbers: the amounts of blue, green and red light. The only difference is the order in which they are stored. OpenCV keeps the order Blue, Green, Red, whereas Matplotlib (and most other libraries) expect Red, Green, Blue. If a BGR array is shown directly with plt.imshow(), the first channel is drawn as red, so red and blue exchange places, as the “Unconverted BGR Display” panel in Figure 4.1 shows. The fix is cv.cvtColor(img, cv.COLOR_BGR2RGB) before displaying, and converting back with cv.COLOR_RGB2BGR before saving with cv.imwrite().

5.2 Resolution versus bit depth

Resolution is how many pixels the image has, width × height. It controls how much spatial detail can be represented, such as how small an object can still be seen. Bit depth is how many bits are used for each pixel value, and it controls how many different brightness (or color) levels each pixel can take: 2^n levels for n bits. In this lab, resizing the crop from 100 × 100 to 200 × 200 changed the resolution (the pixel count) while the values stayed 8-bit. The gradient in Step 7 kept the same width but lost levels (256, 16, 4, 2), which changed only the bit depth.

5.3 What a histogram shows and does not show

A histogram counts how many pixels have each intensity value. It shows the overall brightness, the contrast (how widely the values are spread), and whether many pixels are crowded at the dark or bright end, which would point to under- or over-exposure. It does not show where the pixels are located, so it says nothing about shapes, objects, edges or texture, and a grayscale histogram also hides color. Two very different pictures can have the same histogram, for example an image and the same image with its pixels shuffled.

6. Conclusion

In this lab I loaded a color image with OpenCV and inspected it as an array of uint8 values indexed by [row, column]. I saw why BGR must be converted to RGB for correct display, split the image into channels and a grayscale version, cropped an exact 100 × 100 region and resized it to 200 × 200, and saved the result. The histogram showed the brightness distribution of the image but nothing about where those pixels are. The bit-depth demonstration showed that fewer bits mean fewer gray levels and visible banding, which is a different property from resolution. The script lab02.py runs without errors from inside the virtual environment, and all required files were produced.

7. Submitted Files

- lab02.py
- The input image (my own selected color image)
- resized_crop.png
- grayscale_histogram.png
- .gitignore
- This report: 2024-SET-09-LAB02.docx

Appendix: Complete Source Code (lab02.py)

```
"""
Digital Image Processing Lab Task 02
Digital Images, Pixels, Color, and Basic Transformations

Name   : Syeda Khadija Hassan
Roll No: 2024-SE-09
"""

import os
import sys

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

# ---------------------------------------------------------------------
# Settings (change IMAGE_PATH to the name of your own image file)
# ---------------------------------------------------------------------
IMAGE_PATH = "my_image.jpg"
CROP_Y_START, CROP_X_START = 100, 50     # top-left corner of the crop
CROP_SIZE = 100                          # crop is CROP_SIZE x CROP_SIZE
RESIZE_TO = (200, 200)                   # (width, height) for cv.resize()



def banner(title):
    print("\n" + "=" * 60)
    print(title)
    print("=" * 60)



# ---------------------------------------------------------------------
# Step 1: Verify environment
# ---------------------------------------------------------------------
banner("STEP 1: Environment check")
print("Python executable :", sys.executable)
print("NumPy version     :", np.__version__)
print("OpenCV version    :", cv.__version__)
print("Matplotlib version:", plt.matplotlib.__version__)

# ---------------------------------------------------------------------
# Step 2: Load the image
# ---------------------------------------------------------------------
banner("STEP 2: Load the image")
img = cv.imread(IMAGE_PATH)          # OpenCV loads color images as BGR
if img is None:
    raise FileNotFoundError(f"Image not found or unreadable: {IMAGE_PATH}")

print("Image shape (H, W, C):", img.shape)
print("Image data type      :", img.dtype)

# ---------------------------------------------------------------------
# Step 3: Inspect the image
# ---------------------------------------------------------------------
banner("STEP 3: Inspect the image")
height, width, channels = img.shape
print("Height  :", height, "pixels")
print("Width   :", width, "pixels")
print("Channels:", channels)

center_row, center_col = height // 2, width // 2
```

```
print("Top-left pixel [0, 0] (B, G, R)              :", img[0, 0])
print(f"Center pixel [{center_row}, {center_col}] (B, G, R):", img[center_row, center_col])

print("Global minimum value:", img.min())
print("Global maximum value:", img.max())
for index, name in enumerate(("Blue", "Green", "Red")):   # BGR order
    print(f"{name:>5} channel -> min: {img[:, :, index].min()}, max: {img[:, :, index].max()}")

# ---------------------------------------------------------------------
# Step 4: Convert colors and separate channels
# ---------------------------------------------------------------------
banner("STEP 4: Color conversion and channels")
rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
red, green, blue = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

fig, axes = plt.subplots(2, 3, figsize=(12, 8))
panels = [
    (rgb, "RGB Image", None),
    (red, "Red Channel", "gray"),
    (green, "Green Channel", "gray"),
    (blue, "Blue Channel", "gray"),
    (gray, "Grayscale Image", "gray"),
    (img, "Unconverted BGR Display", None),   # wrong colors on purpose
]
for ax, (data, title, cmap) in zip(axes.flat, panels):
    ax.imshow(data, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()

r, g, b = (int(v) for v in rgb[center_row, center_col])
hex_value = f"#{r:02X}{g:02X}{b:02X}"
print(f"Center pixel RGB value: ({r}, {g}, {b})")
print("Center pixel HEX value:", hex_value)

# ---------------------------------------------------------------------
# Step 5: Crop, resize, and save
# ---------------------------------------------------------------------
banner("STEP 5: Crop, resize, and save")
y_start, x_start = CROP_Y_START, CROP_X_START
y_end, x_end = y_start + CROP_SIZE, x_start + CROP_SIZE

crop = rgb[y_start:y_end, x_start:x_end]
print("Crop shape:", crop.shape)
if crop.shape[:2] != (CROP_SIZE, CROP_SIZE):
    raise ValueError("Crop region falls outside the image; adjust the start coordinates.")

resized = cv.resize(crop, RESIZE_TO)
print("Resized shape:", resized.shape)

fig, axes = plt.subplots(1, 3, figsize=(12, 5))
for ax, (data, title) in zip(axes, [(rgb, "Original Image"),
                                    (crop, "100x100 Crop"),
                                    (resized, "200x200 Resized")]):
    ax.imshow(data)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()

resized_bgr = cv.cvtColor(resized, cv.COLOR_RGB2BGR)
saved = cv.imwrite("resized_crop.png", resized_bgr)
print("cv.imwrite() saved resized_crop.png successfully:", saved)
```

```
# ------------------------------------------------------------------------
# Step 6: Histogram
# ------------------------------------------------------------------------
banner("STEP 6: Grayscale histogram")
flat = gray.flatten()
print("Mean grayscale intensity:", round(float(flat.mean()), 2))
print("Dark pixels   (0-84)   :", f"{np.mean(flat < 85) * 100:.1f}%")
print("Mid pixels    (85-169) :", f"{np.mean((flat >= 85) & (flat < 170)) * 100:.1f}%")
print("Bright pixels (170-255):", f"{np.mean(flat >= 170) * 100:.1f}%")

plt.figure(figsize=(12, 6))
plt.hist(flat, bins=256, range=(0, 256), color="#333333")
plt.title("Grayscale Image Histogram")
plt.xlabel("Pixel Intensity (0-255)")
plt.ylabel("Frequency (Number of Pixels)")
plt.savefig("grayscale_histogram.png", dpi=150, bbox_inches="tight")
print("Histogram saved as grayscale_histogram.png:", os.path.exists("grayscale_histogram.png"))
plt.show()

# ------------------------------------------------------------------------
# Step 7: Bit depth
# ------------------------------------------------------------------------
banner("STEP 7: Bit depth demonstration")
gradient = np.tile(np.arange(256, dtype=np.uint8), (100, 1))   # 0..255 left to right



def quantize(image, bits):
    levels = 2 ** bits
    step = 256 // levels                        # width of each input bin
    return ((image // step) * (255 // (levels - 1))).astype(np.uint8)



versions = [
    (gradient, "Original 8-bit (256 levels)"),
    (quantize(gradient, 1), "1-bit (2 levels)"),
    (quantize(gradient, 2), "2-bit (4 levels)"),
    (quantize(gradient, 4), "4-bit (16 levels)"),
]
fig, axes = plt.subplots(4, 1, figsize=(12, 8))
for ax, (data, title) in zip(axes, versions):
    ax.imshow(data, cmap="gray", vmin=0, vmax=255, aspect="auto")
    ax.set_title(title)
    ax.axis("off")
    print(f"{title:<30} unique values: {len(np.unique(data))}")
plt.tight_layout()
plt.show()

banner("DONE: lab02.py finished without errors")
```